COS 487 / 587 Course Project Part 2: IR Rangers

Esther Greene
University of Southern Maine
Portland, Maine, USA
esther.greene@maine.edu

Courtney Jackson
University of Southern Maine
Portland, Maine, USA
courtney.jackson@maine.edu

Nathaniel Serrano
University of Southern Maine
Portland, Maine, USA
nathaniel.serrano@maine.edu

Abstract

Keywords

information retrieval, BM25, SPLADE, dense retrieval, rank fusion, scientific document retrieval, passage retrieval

1 Introduction

2 Methodology

2.1 Preprocessing

2.2 BM25 Baseline

2.3 Training Data

2.4 Learned Sparse Retrieval

2.4.1 Architecture and Representation. For learned sparse retrieval we use SPLADE [3], initialized from the `naver/splade-cocondenser-selfdistil` checkpoint [2], which also serves as our zero-shot baseline. SPLADE maps a text (query or paragraph) to a sparse vector wR|V| over the BERT WordPiece vocabulary (|V|=30,522). For each input position t, the Masked Language Modeling (MLM) head produces a logit wt,j for every vocabulary term j; these are passed through a log-saturated ReLU and max-pooled over positions (padding masked out):

$$w_j = \max_t \log\left(1 + \mathrm{ReLU}(w_{t,j})\right) \tag{1}$$

The ReLU zeroes out most terms, max-pooling keeps each term's strongest contribution anywhere in the text, and the logarithm damps very large weights so that no single term dominates a score. Because the MLM head scores the whole vocabulary, a text can receive weight on terms it does not contain, which lets SPLADE expand queries and documents with related terms while still matching exact terms. Inputs are truncated at 256 tokens.

2.4.2 Model Fine-Tuning. We fine-tune the base model on the training triples described in Section 2.3 with `SpladeLoss`, which combines `SparseMultipleNegativesRankingLoss` with FLOPS regularization [5]. The ranking loss is a softmax cross-entropy in which each query's positive paragraph must outscore its BM25 hard negative and all other paragraphs in the batch. A no-duplicates batch sampler keeps a query from appearing twice in one batch, so its other positives are never treated as negatives. FLOPS regularization penalizes the sum over the vocabulary of each term's squared mean weight across the batch, which discourages terms that are active in many texts and so keeps posting lists short; we weight it 510-5 for queries and 310-5 for documents.

Fine-tuning settings:

- **Batch size:** 32 (mixed precision, fp16)
- **Learning rate:** 110-5
- **Training horizon:** up to 3 epochs
- **Validation and early stopping:** validation nDCG@10 every 50 steps; training stops after 3 evaluations without improvement, and the best checkpoint is kept.

The best checkpoint (step 300, epoch 1.86) raised validation nDCG@10 from 0.566 to 0.644.

2.4.3 Indexing and Retrieval.

Index construction. Cleaned paragraphs are encoded on the GPU in batches of 64. Each batch's activations are copied to host memory in a single transfer and stored as a small CSR matrix. The batches are then stacked into one paragraph vocabulary matrix and converted to Compressed Sparse Column (CSC) format, so that each vocabulary term's column, holding the paragraphs that contain it and their weights, is contiguous, as in an inverted index. The matrix is saved as a compressed `.npz` file, together with the paragraph IDs in row order and the encoder settings. Retrieval loads this saved index; it is rebuilt only on request.

Search. Queries are encoded by the same pipeline into qR|V|, and a paragraph's score is the sparse dot product

$$S(q,d) = \langle \mathbf{q}, \mathbf{d} \rangle = \sum_{j \in V} q_j \, d_j \tag{2}$$

Only the columns of the query's active terms are read, which works like term-at-a-time scoring over an inverted index. The top k paragraphs are returned with their scores; paragraphs that share no active term with the query score zero and are left out.

2.5 Dense Retrieval

For dense retrieval, we used the pretrained `all-MiniLM-L6-v2` Sentence-BERT bi-encoder [6, 7]. The model encodes queries and paragraphs into 384-dimensional dense embeddings. This lets us have retrieval based on semantic similarity rather than exact term overlap. Embeddings were normalized so that inner-product similarity is equivalent to cosine similarity.

We fine-tuned the model using 5,138 query-positive paragraph pairs with MultipleNegativesRankingLoss, which treats other positive passages within a batch as in-batch negatives. Training used a batch size of 16, a learning rate of 2×10^{-5}, and five epochs. We then performed validation after each epoch, and the checkpoint with the highest validation nDCG@10 was selected. The best checkpoint was obtained after the second epoch.

We indexed 2,275,547 paragraph embeddings using FAISS [4]. Specifically, we used `IndexFlatIP`, which performs exact inner-product search. Each query was encoded using the same bi-encoder, and the index returned the highest- scoring paragraphs based on similarity.

2.6 Paper Ranking

Our neural models index paragraphs only, so paper rankings are built from paragraph rankings with MaxP [1]: each paper is scored by its best-scoring retrieved paragraph,

$$\text{score}(p) = \max_{d \in p \cap R_q} s(q, d), \tag{3}$$

where s(q,d) is the model's score for paragraph d and Rq is the set of paragraphs retrieved for query q. We retrieve the top 2,000 paragraphs, map each to its paper, and walk down the list keeping the first occurrence of each paper; since the list is sorted by score, a paper's first occurrence is its best paragraph. We stop at 100 distinct papers. Retrieval goes far deeper than 100 because one paper often has several paragraphs near the top of the ranking; at a depth of 2,000, every Study query yielded 100 distinct papers. Papers with no paragraph in the top 2,000 are not ranked. The BM25 baseline keeps its own full-paper index; in Section 4.2 we also rank papers with BM25 by MaxP to separate the effect of MaxP from that of neural matching.

2.7 Rank Fusion

$$\text{RRF}(d) = \sum_{m \in M} \frac{1}{k + \text{rank}_m(d)}, \qquad k = 60, \tag{4}$$

2.8 Evaluation Setup

3 Evaluation Results

3.1 Training

For Dense retrieval, we trained the model for five epochs and evaluated its performance after each epoch. The checkpoint from the second epoch achieved the highest validation nDCG@10 and was selected for our final retrieval experiments. Although training continued for all five epochs, the later checkpoints did not improve validation retrieval performance.

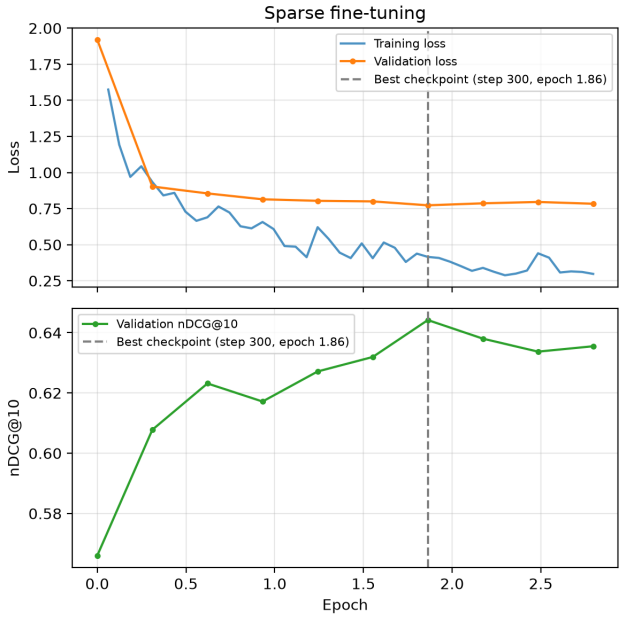
3.2 Effectiveness

For Dense retrieval, we observed different results between paper-level and paragraph-level retrieval. On papers, Dense achieved an nDCG of 0.640 compared to 0.582 for BM25, representing a statistically significant improvement. However, on paragraphs, Dense achieved an nDCG of 0.447 compared to 0.474 for BM25. This difference was not statistically significant.

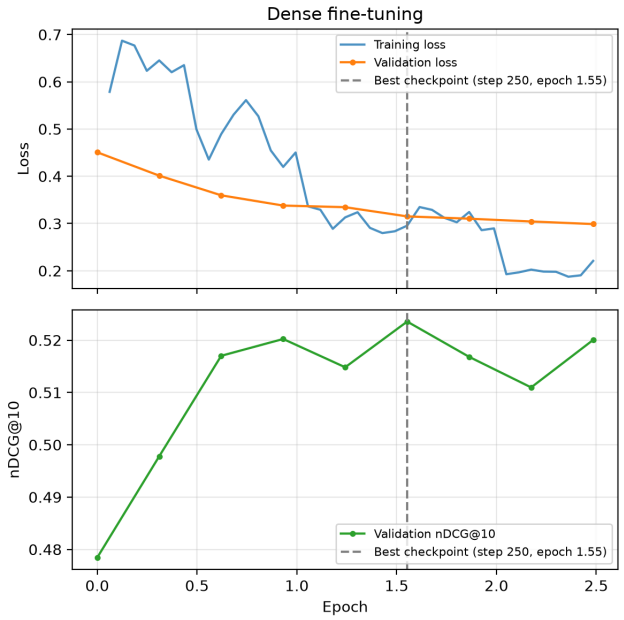
These results suggest that Dense retrieval was more effective at identifying relevant papers than ranking individual relevant paragraphs. One possible explanation is our MaxP paper-ranking approach, which uses the highest-scoring retrieved paragraph to represent each paper.

3.3 Effect of Fine-Tuning

As shown in Table 2, fine-tuning did not lead to significant improvements for either neural model. For Dense retrieval, paper-level nDCG increased from 0.635 to 0.640, while paragraph-level nDCG decreased from 0.458 to 0.447. The zero-shot Dense model also performed significantly better on paper-level P@5, scoring 0.166 compared to 0.156 for the fine-tuned model ($p < 0.05$). This



(a) SPLADE.



(b) Dense (MiniLM).

Figure 1: Training and validation loss.

suggests that fine-tuning did not consistently improve Dense retrieval on our Study queries. Although paper-level nDCG increased slightly, paragraph-level nDCG and paper-level P@5 decreased.

SPLADE showed small improvements after fine-tuning, with paragraph-level nDCG increasing from 0.485 to 0.494. What we see is that these improvements were not really that statistically

Table 1: Effectiveness on the 250 Study queries. Strict metrics count only grade-2 documents (238 queries). Superscripts mark a significant improvement over the model with that letter (paired t-test, $p < 0.05$).

	Model	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	Dense	0.640^{a}	0.592^{a}	0.631^{a}	0.156^{a}	0.084	0.807	0.606^{a}	0.141^{a}
c	SPLADE	**0.671**^{a}	**0.631**^{ab}	**0.658**^{a}	**0.167**^{ab}	**0.090**^{ab}	**0.825**	**0.645**^{a}	**0.151**^{ab}
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	Dense	0.447	0.377	0.477	0.175	0.104	0.607	0.415	0.115
c	SPLADE	**0.494**^{b}	**0.434**^{b}	**0.539**^{b}	**0.190**	**0.116**^{b}	**0.630**^{a}	**0.484**^{b}	**0.135**^{ab}

Table 2: Zero-shot versus fine-tuned neural models on Study queries. † marks a significant difference between the two versions of a model (paired t-test, $p < 0.05$).

	Papers					Paragraphs				
Model	nDCG	nDCG@5	MRR	P@5	R@100	nDCG	nDCG@5	MRR	P@5	R@100
Dense, zero-shot	0.635	0.591	0.619	0.166†	0.812	0.458	0.385	0.484	0.174	0.633
Dense, fine-tuned	0.640	0.592	0.631	0.156	0.807	0.447	0.377	0.477	0.175	0.607
SPLADE, zero-shot	0.668	0.625	0.654	0.161	0.815	0.485	0.431	0.524	0.186	0.603
SPLADE, fine-tuned	0.671	0.631	0.658	0.167	0.825	0.494	0.434	0.539	0.190	0.630

significant. Overall, our results suggest that fine-tuning did not provide a clear advantage over the pretrained models on the Study query set.

3.4 Rank Fusion

3.5 Query Difficulty

3.6 Test Annotation

For each of the 250 Test queries we judged the top-ranked paragraph of our best Study model, fine-tuned SPLADE, using the same 0–2 scale and guidelines as in Part 1: *relevant* (2), *partially relevant* (1), or *non-relevant* (0). Annotators saw the paragraph's original text, including equations, and the paper's title. For 79 queries (32%), SPLADE's top paragraph was the one BM25 had ranked first in Part 1, so we reused its Part 1 grade; the remaining 171 were split evenly among the three of us (57 each), balanced by difficulty.

SPLADE's top paragraph is at least partially relevant for 203 queries (81%), against 197 (79%) for BM25, and fully relevant for 113 (45%), against 104 (42%) (Table 5). The gain comes from hard queries, where SPLADE's top paragraph is relevant for 32 of 50 against 22 for BM25, while BM25 stays slightly ahead on easy queries (70 vs. 66 of 75), the same pattern as in the Study results (Section 3.5). These counts are descriptive: each model has one judged paragraph per query, and the two sets of judgments were made in different phases of the project.

4 Results Analysis

4.1 Comparing the Neural Models

As we see in Table 1, SPLADE performed better than Dense retrieval on both papers and paragraphs. For paper retrieval, SPLADE was able to get and nDCG of 0.671 compared to 0.640 for Dense. For paragraph retrieval, SPLADE also performed better, with an nDCG of 0.494 compared to 0.447. The differences in paragraph-level nDCG were stastistically significant.

One possible explanation is how the two models represent text. SPLADE uses vocabulary-based representations that can preserve important technical terms while also expanding beyond exact word matches Dense retrieval represents each paragraph using a 384-dimensional embedding, which helps capture semantic similarity but may lose some specific technical information.

Dense retrieval also performed better than BM25 on papers, but not on paragraphs. This may be partly due to our paper-ranking approach, where a paper receives the score of its highest-ranked paragraph. This gives Dense more opportunities to identify a relevant paper, even when it does not rank the exact relevant paragraph highly.

4.2 Comparison with the BM25 Baseline

4.3 Successful Retrieval Cases

4.4 Failure Cases

4.5 Why Fine-Tuning Gave Little

One possible reason fine-tuning did not improve our results significantly is that both pretrained models had already learned useful

Table 3: Reciprocal rank fusion on Study queries. Superscripts mark a significant improvement over the run with that letter (paired t-test, $p < 0.05$).

	Run	nDCG	nDCG@5	MRR	P@5	P@10	R@100	Strict MRR	Strict P@5
Papers									
a	BM25	0.582	0.514	0.544	0.137	0.080	0.803	0.535	0.124
b	SPLADE	0.671^{a}	0.631^{a}	0.658^{a}	0.167^{a}	0.090^{a}	0.825	0.645^{a}	0.151^{a}
c	RRF(BM25+SPLADE)	0.675^{a}	0.628^{a}	0.659^{a}	0.170^{a}	0.094^{a}	0.834^{a}	0.646^{a}	0.149^{a}
d	RRF(all three)	$\mathbf{0.701}^{abc}$	$\mathbf{0.655}^{ac}$	$\mathbf{0.689}^{ac}$	$\mathbf{0.174}^{a}$	$\mathbf{0.095}^{ab}$	$\mathbf{0.853}^{abc}$	$\mathbf{0.673}^{ac}$	$\mathbf{0.154}^{a}$
Paragraphs									
a	BM25	0.474	0.416	0.516	0.174	0.107	0.592	0.450	0.115
b	SPLADE	0.494	0.434	0.539	0.190	0.116	0.630^{a}	0.484	0.135^{a}
c	RRF(BM25+SPLADE)	0.531^{ab}	0.468^{ab}	0.569^{a}	0.201^{a}	0.119^{a}	0.666^{ab}	0.508^{a}	0.136^{a}
d	RRF(all three)	$\mathbf{0.550}^{abc}$	$\mathbf{0.489}^{abc}$	$\mathbf{0.582}^{ab}$	$\mathbf{0.218}^{abc}$	$\mathbf{0.130}^{abc}$	$\mathbf{0.689}^{abc}$	$\mathbf{0.523}^{ab}$	$\mathbf{0.148}^{abc}$

Table 4: nDCG@5 on Study queries by difficulty level.

	Difficulty	BM25	Dense	SPLADE	RRF
Papers	Easy	0.635	TODO	TODO	TODO
	Medium	TODO	TODO	TODO	TODO
	Hard	TODO	TODO	TODO	TODO
Paragraphs	Easy	0.676	TODO	TODO	TODO
	Medium	TODO	TODO	TODO	TODO
	Hard	TODO	TODO	TODO	TODO

Table 5: Grades of the top-ranked paragraph for the 250 Test queries, by difficulty: fine-tuned SPLADE (Part 2) and BM25 (Part 1). For 79 queries both models ranked the same paragraph first, and its Part 1 grade was reused.

Grade	Easy	Medium	Hard	All
SPLADE (Part 2)				
Relevant (2)	47	54	12	113
Partially relevant (1)	19	51	20	90
Non-relevant (0)	9	20	18	47
Relevant or partially	88%	84%	64%	81%
BM25 (Part 1)				
Relevant (2)	49	48	7	104
Partially relevant (1)	21	57	15	93
Non-relevant (0)	5	20	28	53
Relevant or partially	93%	84%	44%	79%

representations from large training datasets. Our training set contained around 1,800 queries, which may not have been enough to make a noticeable improvement over the original models. This could be an explanation of why we saw only small changes in retrieval performance after fine-tuning.

Another possible factor is how the training data was prepared. For SPLADE, hard negatives were selected using BM25 rankings, which may have limited the model's exposure to mistakes from its own retrieval results. For Dense retrieval, we used in-batch negatives rather than directly training on the BM25-selected negatives. This may have limited the variety of difficult examples the model encountered.

Also, our relevance judgements were limited, meaning some paragraphs treated as negative examples may have actually been relevant. We attempted to reduce this risk during BM25 negative sampling by skipping the top 20 results. Finally, selecting checkpoints using validation nDCG@10 from a relatively small query set may have contributed to differences between validation performance and our final Study results.

4.6 Why Fusion Helps

Our results suggest that combining different retrieval models can improve performance because each model approaches retrieval differently. BM25 focuses on exact term matching, SPLADE combines term matching with learned expansion, and Dense retrieval uses semantic similarity. Because of these differences, the models may retrieve relevant documents that the others miss.

As we see in Table 3, combining all three models using Reciprocal Rank Fusion (RRF) produced the best overall results. Paper-level nDCG increased from 0.671 for SPLADE alone to 0.701 for RRF, while paragraph-level nDCG increased from 0.494 to 0.550.

Something interesting to note is that adding Dense to the BM25 and SPLADE combination improved paragraph-level nDCG from 0.531 to 0.550, even though Dense had the lowest individual paragraph-level nDCG. This suggests that Dense contributed useful retrieval results that complemented the other two models.

4.7 Strengths and Weaknesses

Each retrieval approach had different strengths and weaknesses. BM25 was useful for matching exact terms, including technical names and acronyms. It was also relatively simple and efficient, but struggled when queries used different terminology from relevant documents.

SPLADE was our strongest individual retrieval model. By combining vocabulary-based matching with learned term expansion, it could retrieve relevant documents even when the query and document did not use exactly the same words. However, it still had limitations when similar terms appeared in unrelated scientific contexts. It also required more computational resources than BM25.

Dense retrieval was useful for capturing semantic similarity between queries and documents, but it performed worse than SPLADE overall. Its compact embeddings may have lost some important technical details, particularly when retrieving specific paragraphs.

Finally, RRF achieved the best overall performance by combining the rankings from all three models. This approach also required running multiple retrieval systems, increasing computational cost compared to using a single model.

5 Conclusion

References

[1] Zhuyun Dai and Jamie Callan. 2019. Deeper Text Understanding for IR with Contextual Neural Language Modeling. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 985–988.

[2] Thibault Formal, Carlos Lassance, Benjamin Piwowarski, and Stéphane Clinchant. 2022. From Distillation to Hard Negative Sampling: Making Sparse Neural IR Models More Effective. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2353–2359.

[3] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2288–2292.

[4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[5] Biswajit Paria, Chih-Kuan Yeh, Ian E. H. Yen, Ning Xu, Pradeep Ravikumar, and Barnabás Póczos. 2020. Minimizing FLOPs to Learn Efficient Sparse Representations. In *International Conference on Learning Representations*.

[6] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 3982–3992.

[7] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33.